

비전 AI 기반 불량 분류 및 원인 추적 Agent — 기술설명서

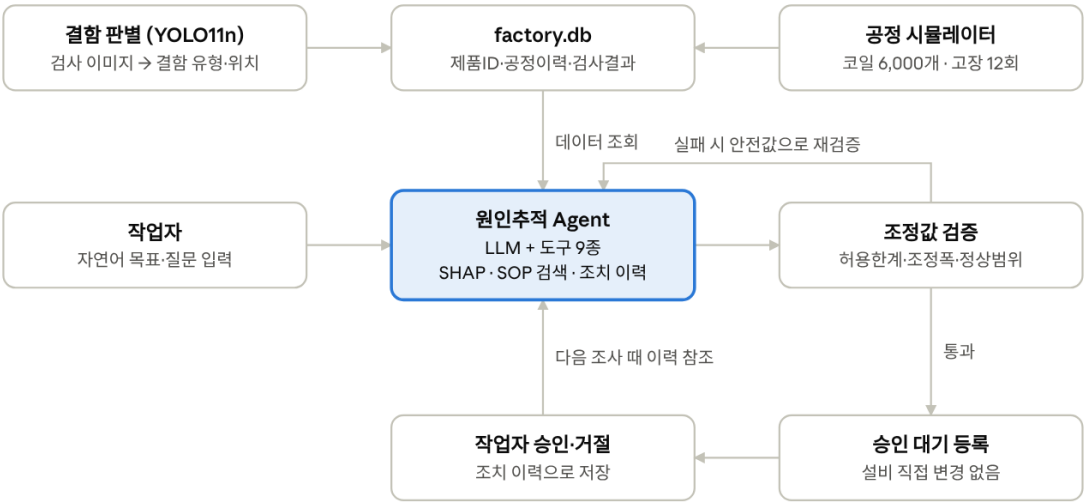
팀명 GPT 가 아니라던데요(김동환·김승록·유동현) · 대회 제 4 회 경남 AI·SW 경진대회 대학부 · 작성일 2026-10-05

이 문서의 공정 데이터, 작업표준서(SOP), 공정 사양값은 모두 시뮬레이션용 가정이며 실제 설비 데이터가 아니다. 공정 관련 성능 수치는 "시뮬레이션 시나리오 기준"으로만 해석한다.

1. 시스템 개요

열연 강판 공장에서 표면 결함이 급증해도 어느 공정의 어떤 설정값 때문인지 찾는 데는 숙련자의 시간이 많이 든다. 본 시스템은 비전 AI로 결함을 판별하고, 제품 ID로 공정 이력을 결합해 원인 공정·변수를 추적한 뒤, 설비 허용범위 안에서 조정값을 근거와 함께 제안한다. 제안은 작업자가 승인해야만 반영되며, AI가 설비를 직접 바꾸지 않는다.

Agent가 조사하고, 조치는 작업자 승인 후에만 반영된다



시스템 구성도 · 비전 → 데이터 → Agent → 검증 → 승인

검사 이미지는 YOLO가 판별해 데이터베이스에 기록하고, Agent는 이 기록과 공정 이력을 조회해 원인을 찾는다. Agent가 낸 조정값은 검증을 통과해야 승인 대기로 등록되고, 승인·거절 결과는 다음 조사 때 참조되는 이력이 된다.

구성요소	역할	구현 위치
비전 모델	강판 이미지의 결함 유형·위치 판별	vision/infer.py, vision/detector.py
공정 시뮬레이터	제품별 공정변수 이력과 고장	scripts/

구성요소	역할	구현 위치
	에피소드 생성	simulate_process.py, config/process_spec.yaml
데이터베이스	제품·공정이력·검사결과· 조치이력 저장 (SQLite)	data/factory.db
원인 분석	급증·이탈 탐지, SHAP 원인 순위	agent/tools.py
AI Agent	규칙·통계 엔진이 원인·조정값을 계산·검증하고, 로컬 LLM(Qwen3 8B)이 도구로 추가 조사해 작업자에게 설명	agent/chat.py(대화), agent/agent.py(터미널 조사 모드)
시연 앱	가상 컨베이어, 검출 보관함, 원인 분석 대화	app.py (Streamlit)

Agent 의 내부 구조와 도구는 별도의 「AI Agent 기술설명서」 에서 다룬다.

2. 데이터

2.1 원본 데이터셋

결함 판별에는 공개 데이터셋 NEU-DET(Northeastern University Surface Defect Database)을 사용했다. 열연 강판 표면을 촬영한 200×200 흑백 이미지로, 결함 6 종이 클래스당 300 장씩 총 1,800 장이며 이미지마다 결함 위치(bounding box)와 유형 라벨이 붙어 있다.

클래스	한글명	학습	검증
crazing	균열(크레이징)	270	30
inclusion	개재물	270	30
patches	패치(얼룩)	270	30
pitted_surface	피팅(점식)	270	30
rolled-in_scale	압입 스케일	270	30
scratches	스크래치	270	30
합계		1,620	180

검증셋은 클래스별 번호 271~300 번 이미지이며, 학습에 쓰지 않고 성능 측정에만 사용한다.

2.2 의사 정상(무결함) 이미지

NEU-DET 은 모든 이미지에 결함이 있어 정상 이미지가 0 장이다. 이 상태로 학습한 모델은 정상 강판을 본 적이 없어, 정상 제품을 결함으로 잘못 판정하는 비율이 높았다. 이를 보완하기 위해 결함 부분을 지운 합성 정상 이미지를 만들었다(Prepare_normals.py).

생성 방법은 각 결함 박스(여유 3px)마다 같은 이미지 안에서 박스와 겹치지 않고 주변 밝기가 가장 비슷한 동일 크기 영역을 찾아, Poisson seamless cloning(OpenCV `seamlessClone`)으로 결함 위치를 덮는 것이다. 대체 영역을 찾지 못한 이미지는 제외했다.

박스 밖 영역이 깨끗한 inclusion 과 scratches 만 소스로 사용했다. pitted_surface 는 박스 밖은 깨끗하지만 박스가 커서 대체할 영역이 부족해 제외했고, crazing 은 박스 밖에도 균열망 질감이 퍼져 있고, patches 와 rolled-in_scale 은 라벨 없는 얼룩·스케일이 남아 있어 제외했다.

구분	inclusion	scratches	합계	학습용	테스트용
의사 정상 이미지 (200×200)	196	121	317	281	36

학습 이미지에서 만든 것은 학습용, 검증 이미지에서 만든 것은 테스트용으로 나눠 데이터 누수를 막았다. 테스트용 36 장은 정상 오검출률 측정에, 학습용 281 장은 YOLO 학습의 배경(라벨 없는) 이미지로 쓰였다. 시연 시에는 시뮬레이션의 정상 제품 5,243 개에 이 317 장을 배정해, 정상 판정까지 포함한 흐름을 재현한다(`scripts/assign_normal_images.py`).

합성 이미지이므로 실제 정상 제품과 분포가 다를 수 있고, 원본 라벨에 없던 작은 결함이 남은 경우도 있다(예: scratches_90). 보고서와 발표에서는 "합성 정상 이미지"로 명시한다. 다른 데이터셋의 정상 이미지는 섞지 않았는데, 섞으면 모델이 결함 대신 카메라·재질 차이를 학습해 성능이 부풀려지기 때문이다.

2.3 증강 데이터 실험 (최종 모델 미채택)

학습 데이터를 늘리기 위해 오프라인 증강 데이터셋(`augment_data.py` → `neu-det-aug/`)을 만들어 실험했다. 원본 1,620 장에 변형 증강 6,480 장, 결함 합성 300 장, 배경 이미지 442 장을 더해 학습셋 8,842 장(원본 대비 5.5 배)으로 구성했다.

변형 증강은 좌우·상하 뒤집기, 확대 크롭, 밝기·대비·감마, 불균일 조명, 노이즈, 블러, JPEG 열화를 확률적으로 적용했다. 압연 방향이 고정된 공정 특성상 90 도 회전은 결함 방향을 왜곡하므로 제외했다. 크롭으로 결함이 40% 이상 잘리면 라벨을 제거하는 등 라벨 품질 검사를 거쳤다.

이 데이터로 YOLO11n 을 학습(60 에폭 설정, 56 에폭에서 조기 종료)한 결과 검증 mAP@0.5 가 0.736 으로 원본만 학습한 초기 모델(0.768)보다 낮았고, 정상 오검출은 36 장 중 12 장(33.3%)이었다. 그래서 최종 모델에는 채택하지 않았고, 최종 모델은 원본 학습셋에 의사 정상 배경 이미지만 더해 학습했다(3 장 참조).

3. 비전 모델

3.1 모델과 학습 설정

Ultralytics YOLO11n(nano)을 COCO 사전학습 가중치에서 전이학습했다. 가장 작은 모델을 택한 것은 컨베이어 속도에 맞춘 실시간 추론과, GPU 없는 시연 노트북에서의 실행을 고려한 것이다.

항목	값
기반 모델	yolo11n.pt (사전학습)
학습 데이터	NEU-DET 학습 1,620 장 + 의사 정상 배경 이미지

항목	값
	281 장
에폭 / 조기 종료	100 / patience 25
입력 크기 / 배치	640 / 32
장비	NVIDIA GeForce RTX 3060 12GB (CUDA 12.6)
최종 가중치	runs/neu_bg_yolo11n/weights/best.pt

배경 이미지는 라벨 없이 학습셋에 넣는다. YOLO 는 이를 "검출할 것이 없는 이미지"로 학습하므로, 정상 표면에서 결함을 찾아내는 오검출이 줄어든다.

3.2 성능

아래 수치는 학습에 쓰지 않은 검증셋 180 장과 의사 정상 테스트 36 장으로 측정했다. 비교를 위해 배경 이미지 없이 원본만으로 학습한 초기 모델(neu_yolo11n, 같은 설정으로 Mac 에서 학습)을 같은 방법으로 측정했다.

지표	초기 모델 (원본만)	최종 모델 (+배경 281 장)
mAP@0.5 (검증 180 장)	0.768	0.776
mAP@0.5:0.95	0.422	0.428
이미지 분류 정확도	95.6%	95.0%
정상 오검출률 (의사 정상 36 장)	58.3% (21 장)	16.7% (6 장)

이미지 분류 정확도는 이미지마다 신뢰도가 가장 높은 박스의 클래스가 정답 클래스와 같으면 정답으로 센다(scripts/eval_vision.py, 신뢰도 임계값 0.25). 박스가 하나도 없으면 미검출로 오답 처리한다.

최종 모델은 정상 오검출률을 58.3%에서 16.7%로 크게 줄였고, mAP 도 소폭 올랐다. 분류 정확도는 0.6%p(180 장 중 1 장) 낮아졌지만 정상 제품을 결함으로 오인해 불필요한 원인 조사를 일으키는 문제가 훨씬 크기 때문에 최종 모델로 채택했다.

최종 모델의 클래스별 분류 정확도는 다음과 같다.

crazing	inclusion	patches	pitted_surface	rolled-in_scale	scratches
90.0%	93.3%	100%	90.0%	96.7%	100%

오답 9 장 중 6 장은 박스가 검출되지 않았고(균열 3, 개재물 2, 압입 스케일 1), 3 장은 피팅을 패치로 잘못 분류했다. 균열은 이미지 전체에 퍼진 질감형 결함이라 경계가 모호해 놓치는 경우가 많고, 피팅은 표면의 점 모양 반점이 패치(얼룩)와 비슷해 혼동한 것으로 보인다.

3.3 추론

vision/infer.py 는 시뮬레이션의 각 제품에 배정된 이미지를 판별해 결과(결함 유형, 신뢰도, 박스 좌표)를 inspections 테이블에 기록한다. 시연 앱은 vision/detector.py 로 컨베이어 위 이미지를

실시간 판별한다. 추론 속도는 RTX 3060 GPU 에서 1 장당 평균 **9.2ms**(p95 11.1ms)로 측정되었다 (runs/eval_vision.json).

4. 공정 시뮬레이션

실제 제철소 공정 데이터는 공개되어 있지 않아, 열연 공정을 모사한 시뮬레이터 (scripts/simulate_process.py)로 제품별 공정 이력을 만들었다. 공정 사양은 config/process_spec.yaml 에 정의했으며, 일반적인 공정 범위를 참고한 가정값이다.

4.1 공정과 변수

제강 → 가열로 → 디스케일링 → 사상압연 → 냉각·권취의 5 개 공정, 9 개 공정변수를 둔다. 변수마다 평균·표준편차, 정상 운전 범위(normal), 설비 허용 한계(hard), 1 회 최대 조정폭(max_step)을 정의했다. Agent 가 제안하는 조정값은 이 값들로 검증된다.

공정	변수	의미	정상범위	1 회 최대 조정폭
제강	inclusion_index	슬래브 개재물 지수	0.6~1.4	조정 불가
가열로	furnace_temp	추출 온도	1190~1240 °C	20 °C
가열로	heating_time	가열 시간	190~230 min	15 min
디스케일링	descale_pressure	분사 압력	165~195 bar	15 bar
사상압연	finish_speed	출측 속도	9.2~10.8 m/s	0.8 m/s
사상압연	roll_force	압하력	16.5~20.5 MN	1.5 MN
사상압연	guide_gap	사이드 가이드 간극	2.6~3.4 mm	0.5 mm
냉각·권취	cooling_flow	냉각수 유량	850~950 m³/h	60 m³/h
냉각·권취	coiling_temp	권취 온도	595~635 °C	20 °C

제강 단계의 개재물 지수는 열연 공정에서 조정할 수 없는 상류 변수로 지정했다. 이 변수가 원인이면 Agent 는 조정 대신 로트 보류와 제강 통보를 제안해야 한다.

4.2 결함·원인 규칙

각 결함 유형은 특정 변수가 한쪽으로 벗어날 때 확률이 오른다고 가정했다. 원인 순위(rank_causes)와 원인 추적 평가(eval_cause.py)는 이 규칙을 쓰지 않고 데이터만으로 원인을 찾는다. 다만 대화 엔진은 작업표준서와 같은 내용인 이 매핑을 표본 부족 시 추정, 조정 방향 판단, 참고 항목 분류에 참조한다.

결함	원인 변수와 방향
압입 스케일	디스케일링 압력 낮음, 가열로 온도 높음
피팅	가열 시간 김

결함	원인 변수와 방향
스크래치	가이드 간극 좁음, 출측 속도 빠름
패치	냉각수 유량 적음
크레이징	권취 온도 낮음
개재물	개재물 지수 높음

4.3 데이터 생성 절차

1. **제품 생산:** 코일 6,000 개를 45~75 초 간격으로 생성하고, 제품마다 9 개 변수 값을 공정별 통과 시각과 함께 기록한다. 값은 평균에 느린 흔들림($AR(1)$, 계수 0.98)과 측정 노이즈를 더해 실제 공정처럼 보이게 했다.
2. **고장 주입:** 처음 300 개는 정상 구간으로 두고, 이후 고장 에피소드 12 개(결함 유형별 2 회)를 넣는다. 에피소드 동안 원인 변수 하나가 구간 앞 30%에 걸쳐 서서히 표준편차의 3.5~5 배만큼 벗어나고, 유지되다가 구간이 끝나면 복귀한다.
3. **결함 발생:** 기본 결함률 2%(6 종 합계)에, 원인 변수가 원인 방향으로 벗어난 정도(z)가 3σ 부근을 지나면서 시그모이드 곡선으로 해당 결함 확률이 늘어나 최대 35%p 까지 오른다.
4. **이미지 배경:** 결함 제품에는 그 유형의 NEU-DET 검증 이미지(학습에 쓰지 않은 180 장)를, 정상 제품에는 의사 정상 이미지 317 장을 배정한다. 이 317 장 중 281 장은 YOLO 학습의 배경 이미지로 쓴 것이라, 정상 제품 5,243 개 중 4,643 개에는 학습에 쓴 이미지가 배정된다. 두 경우의 오검출률은 학습 배경 이미지 17.6%(816 개), 테스트용 이미지 16.3%(98 개)로 비슷했다.
5. **정답지 저장:** 에피소드별 구간·원인 변수·방향을 `fault_episodes` 테이블에 따로 저장한다. Agent 도구는 이 테이블을 조회하지 않으며, 평가 스크립트와 시연 재생 시점 선택(`app.py`, `run_demo.py`)에만 쓰인다.

랜덤 시드(42)를 고정해 어느 PC 에서 실행해도 같은 데이터가 만들어진다. 기본 설정에서 결함 제품은 757 개(12.6%)다.

4.4 데이터베이스 구조

테이블	내용
<code>products</code>	제품 ID, 순번, 생산 시각, LOT
<code>process_log</code>	제품별 공정·변수·값·통과 시각
<code>inspections</code>	배정 이미지, 실제 결함(정답), YOLO 판정 결과·신뢰도·박스
<code>fault_episodes</code>	고장 에피소드 정답지 (평가 전용)
<code>actions</code>	Agent 조치 제안과 승인·거절 이력
<code>detections</code>	시연 앱에서 불량으로 판정된 제품 기록

5. 원인 분석

원인 분석은 네 단계의 통계 도구로 이뤄지며, 모두 `agent/tools.py`에 구현되어 Agent가 호출한다. 모든 분석은 "현재 시점" 이전의 데이터만 사용해, 실제 공장에서 실시간으로 판단하는 상황을 재현한다.

5.1 분석 단계

1. **결함 급증 탐지** (`get_defect_summary`): 최근 300개 제품의 결함 유형별 발생률을 그 직전 1,500개(기준 구간)와 비교한다. 최근 5건 이상이면서 발생률이 기준의 2배 이상(최소 1%)이면 급증으로 판정한다.
2. **공정변수 이탈 탐지** (`detect_deviation`): 최근 300개 제품에서 변수별 평균의 z-score와 정상범위 밖 비율을 계산한다. 범위 밖 비율이 20% 이상인 변수를 이탈 변수로 본다.
3. **결함-공정 이력 결합** (`get_process_history`): 제품 ID로 검사 결과와 공정 이력을 결합해, 해당 결함 제품과 정상 제품의 변수 평균 차이를 표준편차 단위로 비교한다.
4. **SHAP 원인 순위** (`rank_causes`): 최근 최대 2,000개 제품으로 "해당 결함 발생 여부"를 예측하는 Gradient Boosting 분류기를 학습하고, 최근 300개 구간의 결함 제품에 대해 SHAP 값을 계산한다. 결함 확률을 올린 기여만 모아 변수별 비율로 순위를 낸다. 결함 표본이 10건 미만이면 "신뢰도 낮음"을 반환하고 순위를 내지 않는다.

앱 대화의 원인 판단은 이 가운데 SHAP 순위(1·2 순위)와 선택 제품의 공정값을 근거로 하며(「AI Agent 기술설명서」 4.1 절), 터미널 조사 모드는 네 도구를 차례로 호출해 결과를 함께 보여준다.

5.2 평가 방법

`scripts/eval_cause.py`는 고장 에피소드 12개 각각에 대해 시점을 맞춰 분석을 실행하고, 그 결과를 정답지(`fault_episodes`)와 비교한다. 평가 시점은 두 가지다.

- **종료 시점**: 에피소드가 끝나는 시점. 이상이 충분히 쌓인 상태다.
- **조기 시점**: 에피소드 시작 후 60개·40개·20개 제품 시점. 이상이 막 나타나기 시작한 상태로, 시점이 이룰수록 같은 결함 표본이 적다.

평가 시점의 "최근 구간"은 에피소드 시작부터 평가 시점까지(최대 300개)로 잡는다. 앱과 터미널 조사 모드는 항상 최근 300개를 쓰므로, 고장 구간이 짧으면 같은 시점이라도 급증 판정이 다를 수 있다(「AI Agent 기술설명서」 6.3 절).

5.3 평가 결과

최종 비전 모델의 실제 판정 결과(정상 제품 5,243개에 의사 정상 이미지 배정 포함)를 입력으로 측정했다.

평가 시점	급증 탐지율	원인 변수 top-1	top-3	원인 공정 top-1
종료 시점	92% (11/12)	100%	100%	100%
조기 시점 (60개)	75% (9/12)	100%	100%	100%
조기 시점 (40개)	42% (5/12)	92% (11/12)	92%	92%
조기 시점 (20개)	0% (0/12)	50% (6/12)	58%	58%

종료 시점과 60 개 시점에서는 12 개 에피소드 모두 원인 변수를 1 순위로 정확히 찾았다. 시점을 더 앞당기면 40 개 시점 92%, 20 개 시점 50%로 떨어지는데, 이상 초기에는 같은 결함 표본이 적어 SHAP 순위가 불안정하기 때문이다. 같은 결함 표본이 10 건 미만이거나 최근 구간에 없으면 Agent 는 SHAP 대신 규칙 기반 추정임을 밝히고 단발성 가능성을 함께 알린다. 제안서 목표(원인 공정 추적 85% 이상)는 발생 후 40 개 시점부터 충족한다. 급증 탐지는 원인 순위보다 늦게 반응한다. 종료 시점에서 놓친 에피소드 12(개재물)는, 정상 제품의 오검출이 개재물(599 건)과 스크래치(315 건)에 몰려 기준 구간의 개재물 발생률이 높아진 탓에 "기준의 2 배" 조건을 넘지 못했다.

의사 정상 이미지가 개재물·스크래치 이미지에서 결함을 지워 만든 것이라 지운 흔적이 남아 이 두 클래스로 오검출되는 것으로 보인다. 즉 비전 모델의 정상 오검출이 원인 분석의 탐지 단계로 이어진다는 것이 확인되었고, 오검출률을 더 낮추는 것이 전체 성능 개선의 핵심이다.

현재 시뮬레이션은 에피소드마다 변수 하나만 이탈하므로 원인 특정 난이도가 낮다. top-1 100%는 이 조건에서의 결과이며, 여러 변수 동시 이탈이나 원인과 무관한 변수의 이탈이 섞인 상황에서의 성능은 아직 검증하지 않았다.

6. 실행 방법

모든 명령은 Windows PowerShell 기준이며 qc-agent/qc-agent 폴더에서 실행한다. 비전 추론과 규칙 엔진은 GPU 없는 노트북에서도 동작하며, 로컬 LLM 은 RTX 3060 급 GPU 를 권장한다.

6.1 환경 구성

```
git clone https://github.com/fkrlssh/-AI-WS-.git
cd -AI-WS-\qc-agent\qc-agent
python -m pip install -r requirements.txt
```

저장소에는 코드와 최종 가중치(runs/neu_bg_yolo11n/weights/best.pt)가 포함되어 있다. 데이터셋은 용량과 라이선스 때문에 포함하지 않으므로 직접 배치한다.

항목	위치	비고
NEU-DET 데이터셋	data/neu-det/	scripts/prepare_data.py 로 내려받기 가능
의사 정상 이미지	data/pseudo_normal/	Prepare_normals.py 결과의 pseudo_normal 폴더
환경 설정	.env	.env.example 을 복사해 작성, 저장소에 올리지 않음

6.2 로컬 LLM 설정 (Ollama)

Agent 의 기본 LLM 은 공장 내부 PC 에서 동작하는 로컬 LLM 이다. 공정 데이터가 외부로 나가지 않는다.

1. ollama.com 에서 Ollama 를 설치한다.
2. 모델을 내려받는다: ollama pull qwen3:8b
3. .env 에 다음을 적는다.

```
LLM_PROVIDER=ollama
OLLAMA_MODEL=qwen3:8b
# OLLAMA_HOST=http://localhost:11434 (기본값)
```

앱 사이드바에서 로컬 LLM(Ollama)과 오프라인(규칙 기반) 중 Agent 모드를 고를 수 있다. LLM 호출이 실패하면 규칙 기반 엔진의 결과를 그대로 보여준다.

설치 후 `ollama list` 에서 `qwen3:8b` 가 모델 ID `500a1f067a9f`, 크기 5.2GB 로 표시되면 평가에 사용한 모델과 같은 버전이다. 모델 파일을 받을 디스크 공간이 5.2GB 이상 필요하다.

6.3 실행 순서

```
python scripts/simulate_process.py          # 공정 이력 생성 → data/factory.db
python scripts/assign_normal_images.py      # 정상 제품에 의사 정상 이미지 배정
python vision/infer.py --weights runs/neu_bg_yolo11n/weights/best.pt
python -m streamlit run app.py              # 시연 앱 (브라우저 localhost:8501)
```

`infer.py` 는 6,000 장 전체를 판별하므로 시간이 걸리고, 끝날 때까지 진행 표시가 없다. 성능 평가는 다음 스크립트로 한다. `eval_cause.py` 는 결과를 화면에만 출력하고, 나머지는 `runs/` 에 결과 파일을 저장한다.

스크립트	측정 내용
<code>scripts/eval_vision.py</code>	분류 정확도, 1 장당 추론 시간, 정상 오검출률
<code>scripts/eval_cause.py</code>	급증 탐지율, 원인 변수·공정 top-1/top-3
<code>scripts/eval_agent_time.py</code>	검증된 조언 표시 시간, 최종 답변 시간
<code>scripts/eval_agent_quality.py</code>	답변 문제 유형(누락·모순·오조치·느림 등) 자동 집계

6.4 자주 겪는 문제

증상	원인과 해결
<code>pip.exe ...</code> 애플리케이션 제어 정책에서 차단	Windows 스마트 앱 컨트롤이 <code>pip.exe</code> 를 막음 → <code>python -m pip</code> 로 실행
<code>simulate_process.py</code> 용어가 인식되지 않음	파일명만 입력함 → <code>python scripts/simulate_process.py</code>
<code>.env.example</code> 경로를 찾을 수 없음	압축 해제로 같은 이름 폴더가 겹침 → <code>README.md</code> 가 있는 가장 안쪽 폴더로 이동
<code>infer.py</code> 가 멈춘 것처럼 보임	진행 표시 없이 작업 중 → 중단하지 말고 완료 대기
앱에서 로컬 LLM 답변이 엔진 원문으로만 나옴	Ollama 미실행 또는 모델 미설치 → <code>ollama pull qwen3:8b</code> 후 Ollama 실행 확인
<code>git pull</code> 시 untracked 파일 덮어쓰기 오류	저장소에 새로 들어온 파일과 같은 이름의 로컬

증상	원인과 해결
	파일 존재 → 해당 파일을 다른 곳으로 옮긴 뒤 다시 pull

7. 한계 및 향후 과제

구분	현재 한계	향후 과제
데이터	공정 데이터·SOP 가 시뮬레이션 가정값	실제 설비 데이터로 공정 사양과 결함-원인 규칙 검증
정상 이미지	합성 이미지라 개재물·스크래치로 오검출되는 흔적이 남음 (오검출률 16.7%)	정상 이미지 품질 개선, 오검출이 많은 두 클래스 집중 보완
원인 분석	에피소드당 변수 1 개 이탈이러 난이도가 낮음	다변수 동시 이탈, 원인과 무관한 이탈이 섞인 시나리오 평가
급증 탐지	기준 구간 오검출이 많으면 급증을 놓침 (탐지율 92%)	오검출 반영한 기준률 보정, 탐지 조건 튜닝
실행 환경	로컬 LLM 은 GPU 가 있어야 실용적인 속도가 나옴	시연 PC 사양에 맞춘 모델 크기 선택, 추론 속도 재측정
SOP 검색	문자 n-gram TF-IDF 검색	문서가 늘어나면 임베딩 기반 벡터 검색으로 교체 (인터페이스 유지)